TECHNICAL AUDIT & CODEBASE REVIEW

Resonance

Premium Meditation & Sound Therapy App — Production-Readiness Audit for iOS & Android Release

Expo SDK 54 · New Architecture | React Native 0.81.5 / React 19.1 | Reanimated 4.1 | expo-audio + react-native-audio-api | Node.js / Express (TS) | PostgreSQL + Drizzle ORM | TanStack React Query | Clerk Auth | pnpm monorepo / EAS Build

---

Prepared for: Nicolas

Date: August 17, 2026

Scope: artifacts/mobile (Expo app), artifacts/api-server (Express/Drizzle backend), artifacts/resonancia-admin (web admin), and repo-wide build/monorepo configuration

Method: Static code audit across four parallel focus tracks — Audio Architecture, Database & Sync, Streak & Progress System, and Geometrix/Build Pipeline — followed by cross-track synthesis

Contents

0 Scope Note & Audit Caveat

> **This checkout is not a runnable monorepo as exported.** There is no root `pnpm-workspace.yaml` or root `package.json`; `artifacts/` contains eight independently Replit-exported apps (each carrying its own `.replit-artifact` marker). Four shared packages referenced everywhere via `workspace:*` — `@workspace/db` (the actual Drizzle schema/migrations), `@workspace/api-zod`, `@workspace/api-client-react`, and `@workspace/premium` — have no source present in this export. `node_modules` is not installed anywhere. This is itself a Critical finding (§5) and it means: the literal Drizzle schema could not be read directly — table/column shapes in §3 are inferred from live query call-sites in `api-server/src/routes/*.ts`, not from the schema file itself. Everything else in this report is a direct static read of the code that *is* present.

1 Executive Summary

The codebase reflects fast, iterative, partly AI-assisted development on a genuinely ambitious app — 200+ screens, a custom gapless BPM audio mixer, a real-time-ish community/DM layer, and an elaborate generative-geometry visual system. Two things are true at once: **some of the hardest engineering in this app (the BPM loop engine, the like/report transaction logic, the Geometrix carousel memoization) is done with real care and already reflects lessons learned from past bugs.** And **the app is not production-ready today** — it ships at least one confirmed live bug that fakes user data on the home screen, an unauthenticated endpoint that lets anyone inflate content rankings, an unresolvable dependency tree as exported, and a systemic pattern of logic duplicated 3–6 times across near-identical screens with no single source of truth.

Area	Verdict	Headline risk
Audio Architecture	Mixed	Three separate libraries (`expo-audio` , `expo-av` , `react-native-audio-api`) independently manage the native audio session; one prior SIGABRT crash from this is fixed by convention, not architecture. BPM engine itself is well-built.
Database & Sync	Not ready	Unauthenticated like-spam endpoint; three list endpoints with no pagination; mix-like cache bug causes visibly stale UI; two disconnected "favorites" data models.
Streak & Progress	Live bug	Hardcoded `DEV_STREAK = 3` ships on the production home screen — every user sees a fake streak. Six divergent streak algorithms exist across the app; no server-authoritative computation at all.
Geometrix (Reanimated)	Good	Both previously-documented carousel bugs are correctly fixed and match team's own tribal-knowledge notes. Minor: one component still uses legacy `Animated` instead of Reanimated.
Monorepo & Build	Broken as exported	No root workspace manifest; 4 shared packages missing entirely; `catalog:` and `workspace:*` references cannot resolve; Metro config hardcodes paths from a specific Replit pnpm store.

Bottom line

This is not a codebase that needs to be thrown out — the audio engine, the transactional backend writes, and the Geometrix rendering pipeline are all evidence of real engineering skill applied under time pressure. What it needs is **consolidation, not reinvention**: pick one audio library, one streak algorithm, one folder-screen component, one response envelope shape — and enforce it. In parallel, it needs the monorepo to be reconstituted into something that actually installs and builds outside of a live Replit session before any release engineering can begin.

2 Audio Architecture

Scope note: no native `android/` / `ios/` generated project or installed `node_modules` was available, so generated manifest/plist output could not be verified directly — flagged inline where relevant.

2.1 Three libraries independently own the native audio session

High **expo-audio, expo-av, and react-native-audio-api all touch AVAudioSession/AudioManager**

`context/PlayerContext.tsx, context/AmbientPlayerContext.tsx, hooks/useVozInterior.ts, lib/bpmAudioEngine.ts:21–23`

`expo-av` (deprecated by Expo in favor of `expo-audio` / `expo-video`) is still actively used in 5 files (ambient scenes, onboarding fade-in, chat voice notes, playlist preview, voice-memo playback) while `expo-audio` is used in 8 files for the core session/mixer player. `hooks/useVozInterior.ts` uses *both* in the same file — recording via expo-audio, playback via expo-av. A real SIGABRT crash caused by configuring the audio session too early (documented in code comments) was fixed by deferring session setup independently in three separate files, rather than through one shared gate — meaning a future contributor unaware of this history can reintroduce the exact crash.

2.2 Duplicated crossfade/loop engines

High **Two independent hand-rolled two-layer crossfade implementations**

`context/PlayerContext.tsx:1026–1134 (session loops) · context/MixerContext.tsx:922–1173 (mixer fallback)`

Each is ~150–200 lines of interval-based gain math with its own drift-correction and seed-confirmation state machine. A precision or glitch fix discovered in one will not automatically propagate to the other. The `expo-audio` fallback path used when the native `react-native-audio-api` module isn't compiled (Expo Go, stale dev client) is documented in the code's own comments as always producing an audible gap at the loop boundary — with no UI indication to the user that they're on the degraded path.

2.3 Chat audio bypasses the app's exclusivity contract

Medium **Voice-message playback in chat doesn't stop session/mixer audio**

`app/chat/[userId].tsx:1098–1243 (AudioAttachment)`

Every other audio surface in the app (session ↔ mixer ↔ descanso sounds) is coordinated through a shared `audioBridge.ts` mutual-exclusion mechanism so only one thing plays at a time. Chat's per-message `expo-av` `Audio.Sound` instances don't register with it — a user can have a meditation session playing and a chat voice note playing simultaneously. Combined with no pooling, a voice-heavy chat thread creates many concurrent native decode operations purely from list scroll/recycle.

2.4 The gapless BPM engine itself is well-engineered

> `lib/bpmAudioEngine.ts` is the standout file in this subsystem: sample-accurate `loopEnd` trimming before trailing silence, token-based cancellation to avoid orphaned-audio races on rapid track switching, proper crossfades on voice replacement, defensive `TurboModuleRegistry` gating before any native call, and a complete `dispose()`. This is not AI-boilerplate — it reads as carefully tuned against real device behavior and should be preserved as-is through any redesign.

2.5 Cleanup & hygiene

Finding	Sev	Location
Sleep-timer logic copy-pasted near-verbatim 3 times instead of a shared hook	Med	PlayerContext.tsx, MixerContext.tsx, AmbientPlayerContext.tsx
19 `eslint-disable exhaustive-deps` across audio files — deps warnings silenced rather than resolved	Med	audio context/hooks, app-wide
Stale 1,783-line `MixerContext.bak` committed alongside the live 91KB file	Low	context/MixerContext.bak
Fade-out `setInterval` in onboarding has no unmount-cancel path	Low	app/onboarding.tsx:275-289
Android background-audio foreground-service manifest config unverifiable (no native project in checkout)	Med — verify	app.json:20-49

Verdict: Mixed — consolidate, don't rewrite

Keep the BPM engine and the crossfade math. Migrate everything off `expo-av` onto `expo-audio` alone (removes an entire class of session-conflict risk), extract the duplicated crossfade/sleep-timer logic into shared hooks, and route chat voice playback through the existing `audioBridge` coordinator.

3 Database, Sync & API Layer

3.1 Critical: unauthenticated, unlimited like endpoint

Critical **/messages/:id/like has no auth and no per-user idempotency**

`api-server/src/routes/messages.ts:120–148`

Does a bare `likes = likes + 1` update with no `requireAuth`, no per-(user,message) uniqueness, and no rate limit — contrast with `mixes.ts`'s like endpoint, which uses a real join table, a transaction, and a row lock. Any anonymous caller can spam this endpoint to arbitrarily inflate a message's rank on `/messages/top`. This is directly exploitable and should be the first fix made.

3.2 Missing pagination on public list endpoints

High **Catalog, followers, following, and friends endpoints return unbounded result sets**

`catalog.ts:219–260 (GET /catalog) · follows.ts:119–183 · friends.ts:44–72`

`GET /catalog` does a full unfiltered scan of all published sessions on every cold app start with no `.limit()` — this is called from `CatalogContext` on mount. `mixes.ts`, by contrast, is properly paginated (page/pageSize/count), showing the team knows the pattern; it just wasn't applied consistently.

3.3 Mix-like optimistic cache bug (real, user-visible)

High **Copy-pasted optimistic update hardcodes the unfiltered query key**

`app/mezcla/[id].tsx:106–122 · components/CommunityMixesCarousel.tsx:66–81`

Both independently call `getGetSharedMixesQueryKey()` with no parameters when patching the like-state cache. Category-filtered lists (`mezclas-comunidad.tsx`) and per-author lists (`mezcla-creador/[userId].tsx`) use parameterized keys and are therefore **not** patched — users see stale like counts/heart state when navigating between these screens until the next refetch. Root cause: the optimistic-update function was copy-pasted rather than extracted into one shared `useMixLike()` hook.

3.4 Two disconnected "favorites" / "playlist" data models

High **Server-synced favorites vs. local-only favorite folders never reconcile**

`api-server/src/routes/activity.ts:104-143 (favoritesTable) · context/FoldersPlaylistsContext.tsx (AsyncStorage-only)`

There are two unrelated concepts both called "favorite": a server-backed table synced cross-device, and a client-only `FavFolder` system persisted purely to AsyncStorage that never touches the server. A favorite added via one path is invisible on another device if added via the other. Similarly, **user-created playlists have no backend table at all** — the only server-side "playlist" concept is admin-curated home-screen content, an unrelated use of the same word. This needs a product decision, not just a code fix: should user playlists/favorite-folders sync to the server?

3.5 React Query: no coherent caching policy

High **QueryClient has zero default options; every screen guesses its own staleTime**

`app/_layout.tsx:89`

Observed `staleTime` values range from `0` to 10 minutes with no shared policy. No offline query persister exists anywhere (`persistQueryClient` / `createAsyncStoragePersister` not found) — all server data is lost on cold start and must be refetched, while a separate hand-rolled AsyncStorage layer exists for local-only playlist data, so the app effectively runs two disconnected persistence models. Several screens poll instead of relying on cache invalidation, most notably chat polling for new messages every **1.5 seconds** — a brute-force simulation of real-time messaging that will cost battery and backend load on any screen left open.

3.6 Duplicated cross-cutting logic across route files

Pattern	Duplicated in	Sev
`toProfile()` serializer, re-implemented per file with drifting field sets	follows.ts, friends.ts, catalog.ts, mixes.ts, notifications.ts, dm.ts (6×)	Low
"Check for existing unread notification, then insert + push" dedup logic	follows.ts, friends.ts (×2), dm.ts — only mixes.ts factored it out	Med
Inconsistent response envelopes: `{mixes,total,...}` vs bare arrays vs `{sounds}`	Across nearly every list endpoint	Med
friend-request creation is check-then-act with no transaction, unlike sibling routes in the same file	friends.ts:131-217	Med

> **What's actually solid:** the write paths that matter most for data integrity — mix likes, mix reports/auto-hide, catalog submission workflow, favorites replace-all sync — correctly use transactions, row locks, and conflict-safe upserts. No SQL injection risk was found anywhere; all raw SQL uses Drizzle's parameterized tagged templates. Error handling (try/catch + structured logging) is consistent across every route file reviewed.

Verdict: Not production-ready as-is, but the fixes are mostly surgical

Priority order: (1) fix/remove the unauthenticated like endpoint, (2) paginate catalog/follows/friends, (3) fix the mix-like cache bug or switch to key-prefix invalidation, (4) set sane QueryClient defaults and replace the 1.5s chat poll with real invalidation or a push channel, (5) make an explicit product decision on favorites/playlist sync.

4 Streak & Progress System

Confirmed live bug, ship-blocking: `app/(tabs)/inicio8.tsx:348–350` — the active production home screen — renders `DEV_STREAK = 3` whenever it's greater than zero, which is always. **Every user sees a hardcoded streak of 3 regardless of real activity.** Git history shows this was an explicit temporary testing commit ("Set the meditation streak counter to three days for testing") that was never reverted. A second identical pattern (`DEBUG_STREAK = 4`) exists in `components/WaveStreakStrip.tsx:209` , currently dormant only because that screen (`inicio6.tsx`) is disabled — it will resurface the instant that screen is re-enabled.

4.1 Six divergent reimplementations of "streak"

High **No single source of truth for streak calculation — and the definitions actively disagree**

`utils/stats.ts:19–38` · `app/progreso.tsx:55–74` · `app/(tabs)/inicio8.tsx:332–346` · `components/SonicStreakWave.tsx, WeeklyStreakStrip.tsx, WeeklyStreakStrip4.tsx, WaveStreakStrip.tsx`

Three components implement "consecutive days ending today/yesterday." Four other components implement an entirely different definition — "days this calendar week meeting a 5-minute threshold" — and also call it a streak. A user who meditates Monday, skips Tuesday, meditates Wednesday sees genuinely contradictory numbers on different screens of the same app. This is a real product-correctness bug, not just a maintainability smell, and the pattern (new home-screen variant → inline-copy the algorithm instead of importing `utils/stats.ts`) is a textbook signature of fast iterative AI-assisted screen generation without a consolidation pass.

4.2 No server-authoritative computation anywhere

High **Streaks and achievements are 100% client-computed and trivially spoofable**

`app/progreso.tsx:127–170 (buildChallenges)` · `api-server/src/routes/activity.ts:64–102`

The backend accepts client-submitted play events (`minutes` , `completed` , `playedAt`) with no plausibility validation — no cap on minutes, no rejection of future timestamps. Since every streak/challenge value is derived client-side from these same events, any user can fabricate activity to inflate their streak or unlock "achievements." There is no achievements/unlock table on the server at all. The UI itself already says "badges coming in future versions," suggesting the team knows this layer is incomplete.

4.3 Local-time-only date math, no grace period

Medium **All streak day-boundary math uses device local time with no timezone anchor or streak-freeze concept**

`All streak calculation sites`

Every implementation buckets days via local `Date` methods with no server/UTC reconciliation. A user who travels, or has a misconfigured device clock, can see a streak silently break or extend inconsistently. There's no "streak freeze"/grace-day allowance anywhere — a single missed local calendar day zeroes the count unconditionally, a common source of user complaints in habit-tracking products.

4.4 What's actually good here

> The cloud-sync merge protocol (`lib/cloudSync.ts`) that reconciles local AsyncStorage progress data with the server on login is genuinely well-designed and self-documented: union-on-first-sync to recover cloud history after reinstall, local-authoritative afterward so deletions persist, with its known limitation (no per-item tombstones/last-write-wins across concurrent multi-device edits) explicitly called out in comments rather than silently present. This is the strongest-engineered piece of the progress system and should be preserved.

Verdict: Needs redesign, not a bug-fix pass — but ship the hotfix first

Step 1 (today): delete both hardcoded debug overrides — a one-line fix, ship-blocking. Step 2 (redesign): move streak/challenge computation to be server-authoritative, computed from the validated activity table on read, returned as one canonical value; delete the six client reimplementations and replace with a single call to that value, with local cache used only as an offline-display fallback clearly marked provisional.

5 Geometrix, Monorepo & Build Pipeline

5.1 Monorepo is not functional as exported — Critical

Critical **No root workspace manifest; four shared packages entirely absent; unresolvable version references**

`repo root · artifacts/mobile/tsconfig.json:18–22 · metro.config.js:46,71–78`

No `pnpm-workspace.yaml` or root `package.json` exists. Every one of the 8 `artifacts/*` subdirectories carries its own `.replit-artifact` marker, confirming each was independently exported from a Replit multi-app workspace rather than checked out as one coherent repo. `@workspace/db` (the real Drizzle schema), `@workspace/api-zod`, `@workspace/api-client-react`, and `@workspace/premium` are depended on via `workspace:*` everywhere but have no source anywhere in this checkout — `pnpm install` cannot resolve them. Every `package.json` in the repo also uses `catalog:` version references, which require a catalog block in a workspace manifest that doesn't exist. `metro.config.js` hardcodes absolute-relative paths into a specific pnpm-hashed package store (`../../node_modules/.pnpm/react-native@0.81.5_...`) two directories above `artifacts/mobile` — this only works inside the original Replit environment (`/home/runner/workspace/`), confirmed independently by the team's own `.agents/memory/metro-config-clobbered.md` notes.

> **Practical implication:** nobody can `pnpm install && expo start` this checkout today, on any machine, including CI. Before any other work on this codebase proceeds at a new engineering org, the monorepo needs to be reconstituted: a real `pnpm-workspace.yaml` + catalog, and the source for all four `@workspace/*` packages, either recovered from the original Replit workspace or rebuilt from what's inferable from call-sites.

5.2 react-native-audio-api forces a fragile Metro resolver patch

High **A native dependency conflict is papered over with hardcoded path-string rewriting instead of fixed at resolution level**

`metro.config.js:64–70,138–151`

`react-native-audio-api` transitively pulls a second React/React Native tree (RN 0.86 + React 19.2) alongside the app's pinned RN 0.81.5/React 19.1. The current fix is a custom Metro `resolveRequest` that string-rewrites package paths — the team's own memory notes confirm this has already silently broken once when a pnpm store hash changed. This should be a `pnpm.overrides`/resolution-level fix, not a bundler-time patch.

5.3 Geometrix / Reanimated performance — the good news

> Both previously-documented Geometrix bugs (`geometrix-carousel-memo`, `geometrix-carousel-microlag`) were independently verified as **correctly fixed in current code**, matching the team's own tribal-knowledge notes almost line-for-line: `GeometrixCarousel` is properly isolated in `React.memo` with stable callback props, and no `transform: scale` wrapper exists around SVG tiles inside the ScrollView (the

> specific pattern known to cause iOS compositing microlag). `CanvasLayer` 's memoization via a referential-identity-cached `getStableSettings` is also correctly implemented and consistently used at every canvas render site. This is a rare, genuinely encouraging case of past debugging effort holding up under new code changes.

Medium **GeoUniverseBackground.tsx still uses legacy RN Animated instead of Reanimated**

`components/GeoUniverseBackground.tsx:8-9,30-92`

Not a functional bug (native driver is used), but it's a second, parallel animation system inside a codebase otherwise standardized on Reanimated — it can't share worklets or gesture-driven shared values with the rest of Geometrix, and represents inconsistency worth cleaning up opportunistically.

Medium **app/(tabs)/geometrix.tsx is a single 7,307-line file**

`app/(tabs)/geometrix.tsx`

Internal memo boundaries are correctly implemented, so this isn't a current performance bug — but it's a long-term maintainability risk: the entire screen, canvas, carousel, and multiple settings sheets/modals are co-located, making review and safe editing difficult.

5.4 Duplication at scale: ~5,000–6,000 lines of redundant/dead screen code

Family	Files	Pattern
Home screen variants	inicio5.tsx, inicio6.tsx (dead, `href: null`), inicio8.tsx (live)	~3,972 lines of superseded iteration kept in the tree instead of git history
Folder detail screens	carpeta/[id].tsx, carpeta-favorito/[id].tsx, carpeta-mezcla/[id].tsx, carpeta-video/[id].tsx	Same UI chrome/interaction logic copy-pasted 4× with only the data-source hook swapped, instead of one parametrized component

This is a sample, not an exhaustive scan — the pattern (copy screen, swap the domain noun) is predictable enough that a full repo-wide near-duplicate scan across all `app/**/*.tsx` would likely surface more.

5.5 Repo hygiene: stray backup files

`.easignore.backup` , `metro.config.js.bak` , `metro.config.js.broken` , `app.json.backup` , `context/MixerContext.bak` (1,783 lines) — all committed into the working tree rather than relying on git history for rollback safety. One config-drift risk found directly: the `.easignore.backup` excludes `artifacts/` from EAS builds while the active `.easignore` does not — evidence of unresolved build-config churn.

Verdict: Monorepo — critical and blocking. Geometrix — in genuinely good shape.

These two halves of this section point in opposite directions and should be read that way: the visual/animation engineering is trustworthy and doesn't need a redesign; the build/dependency

foundation underneath it needs to be rebuilt into something installable before release engineering (EAS builds, CI, App Store submission) can even begin.

6 Redesign vs. Bug-Fix Recommendations

Sorted by what kind of engineering effort each area needs — a quick patch, or a structural rework.

🩹 Simple bug fixes

- Remove `DEV_STREAK` / `DEBUG_STREAK` hardcoded overrides (inicio8.tsx, WaveStreakStrip.tsx)
- Add `requireAuth` + per-user idempotency to `/messages/:id/like`
- Add `.limit()` /pagination to `/catalog`, `/followers`, `/following`, `/friends`
- Fix mix-like optimistic cache to use the correct parameterized query key (or switch to `invalidateQueries` by key-prefix)
- Wrap `friends.ts` request creation in a transaction
- Set `QueryClient` default options (staleTime/gcTime/retry policy)
- Delete stray `.bak` / `.backup` / `.broken` files; resolve the `.easignore` vs `.easignore.backup` discrepancy
- Cancel the onboarding fade-out interval on unmount

🏗️ Needs structural redesign

- **Rebuild the monorepo foundation:** root `pnpm-workspace.yaml` + catalog, recover/rebuild the 4 missing `@workspace/*` packages
- **Server-authoritative streak/achievement engine** — replace 6 client algorithms with 1 computed value from the server
- **Consolidate audio onto expo-audio only** — retire expo-av, unify the two crossfade engines into one shared module
- **Unify favorites/playlists** into one server-synced model (product decision required first)
- **Parametrize the 4 carpeta-* screens** into one component with a content-type prop
- **Fix the react-native-audio-api dependency conflict** at the pnpm-resolution level, not via Metro path patching
- Extract shared route helpers (profile serializer, notify-dedupe, response envelope) into `lib/`

7 Roadmap & Work Plan

Milestone-based plan assuming a small team (2 mobile engineers, 1 backend engineer, part-time QA). Timelines are calendar estimates, not effort-hours, and assume the monorepo is unblocked first since almost everything else depends on it.

Milestone 0 — Unblock the build

3–5 days

- Recover or rebuild `@workspace/db`, `@workspace/api-zod`, `@workspace/api-client-react`, `@workspace/premium` from the original Replit workspace (or reconstruct schema/contracts from route call-sites if the originals are truly gone)
- Create root `pnpm-workspace.yaml` with a `catalog:` block covering React/RN/Zod/TanStack/etc.
- Get `pnpm install` + `expo start` + `pnpm build` (api-server) working from a clean clone, off Replit
- Resolve the react-native-audio-api / React version conflict via `pnpm.overrides` instead of the Metro path hack
- **Exit criteria:** a fresh clone builds and runs on a clean machine with no manual path patching

Milestone 1 — Ship-blocking hotfixes

2–4 days (parallel with M0)

- Remove hardcoded streak debug overrides
- Lock down the unauthenticated message-like endpoint
- Add pagination to the unbounded list endpoints
- Fix the mix-like cache-key bug
- **Exit criteria:** no known data-integrity or fabricated-UI bug remains live

Milestone 2 — Streak & progress redesign

2–3 weeks

- Design + implement server-side streak/achievement computation (endpoint + validation of submitted activity events)
- Migrate all 6 client screens onto the single server value; delete duplicate algorithms
- Add timezone-aware day-boundary logic and a grace-period/streak-freeze policy
- Backfill/reconcile existing users' historical activity into the new computation
- **Exit criteria:** streak/achievement values are identical across every screen and cannot be spoofed by client-submitted data alone

Milestone 3 — Audio consolidation

2–3 weeks

- Migrate expo-av call sites (ambient, onboarding, chat voice notes, voz-interior, playlist preview) onto expo-audio
- Extract one shared crossfade/loop module used by both session player and mixer
- Route chat audio through the existing `audioBridge` exclusivity coordinator
- Verify Android foreground-service manifest output on a real prebuild; confirm background playback on-device (iOS + Android)
- **Exit criteria:** single audio library in use; no known session-conflict crash path; verified background playback on both platforms

Milestone 4 — Data layer cleanup

2 weeks

- Decide and implement unified favorites/playlist sync model
- Standardize list-endpoint response envelope and pagination convention across all routes
- Extract shared profile-serializer and notify-dedupe helpers
- Set QueryClient defaults; replace chat's 1.5s poll with proper invalidation (or a push/WS channel if real-time chat is a priority)
- **Exit criteria:** consistent API conventions; no polling interval under ~10s outside of genuinely real-time surfaces

Milestone 5 — Duplication & cleanup pass

1–2 weeks

- Delete dead `inicio5` / `inicio6` screens (or archive via git tag, not in-tree)
- Parametrize the 4 carpeta-* screens into one component
- Run a repo-wide near-duplicate-file scan to catch additional instances beyond the two sampled families
- Remove all `.bak` / `.backup` / `.broken` files from the tree
- Migrate GeoUniverseBackground.tsx to Reanimated for consistency
- **Exit criteria:** no known dead screen code ships in the bundle; single implementation per screen family

Milestone 6 — Release hardening & store submission

2–3 weeks

- Full regression pass on EAS build profiles (development/preview/production) from the reconstituted monorepo
- Fix `runtimeVersion` policy if OTA updates are intended to be re-enabled
- Device-matrix QA pass focused on background audio, lock-screen controls, and the BPM mixer under real network/battery conditions
- App Store / Google Play submission prep (privacy manifest, permission strings, screenshots, review notes for background audio usage)
- **Exit criteria:** production build submitted to both stores

> **Total estimated timeline: ~10–14 weeks** end-to-end for a small team working roughly in the sequence above, with Milestones 2–5 substantially parallelizable across mobile/backend engineers once Milestone 0 unblocks the build. This is a planning estimate based on the scope of findings in this audit, not a formal engineering estimate — validate against actual team velocity once M0 is complete and the codebase can be built/tested directly.
